Homework 1: Backpropagation and Neural Networks

Deep Learning (84100343-0)
Autumn 2026
Tsinghua University

Introduction

Build a small language model with a team of experts. Shakespeare and Python follow different patterns, yet both can be modeled by predicting what comes next. Must every token use the same feedforward network? You will build a mixture-of-experts (MoE) model and investigate how it can increase capacity while controlling computation per token.

An MoE layer stores several MLPs, called *experts*. A learned *router* selects a few for each token and combines their outputs, separating *total capacity* from *active computation*. Experts learn jointly; their roles are not assigned by hand. Switch Transformers explored this principle at scale [1]. Mixtral 8x7B selects two of eight experts per token for language, code, and multilingual tasks [2]. DeepSeek-V3 reports 671 billion total parameters with 37 billion activated per token [3]. We will study this mechanism at coursework scale.

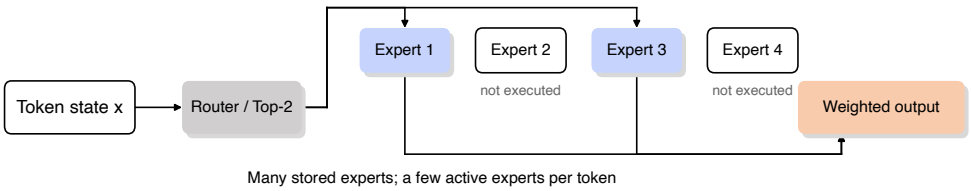


Figure 1: A token activates a small subset of the stored experts. An original schematic of the routing pattern in Mixtral [2], Fig. 1; four experts are shown for clarity.

Our next-byte predictor connects three steps: deriving the building blocks, verifying them in NumPy, and testing a tiny causal language model. Experts use familiar FC layers and ReLU; the causal context module is provided. Please observe the following guidelines:

- Answer item-by-item in your **written report** (PDF), in English or Chinese, and provide readable, commented code. Do not share or submit anyone else's work.
- 📝 questions require **independent handwritten derivations**. Photograph or scan your work and insert it into the PDF. AI-generated derivations are not permitted; a calculator may be used for numerical approximations.
- Unmarked questions require your own core code; AI may explain general syntax or environment errors, but may not supply that code. 🤖 questions encourage AI-assisted exploration. Disclose assistance and discussions; do not invent results.

Total: 100 points (35 handwritten, 50 core programming, 15 exploration). Code and data accompany this PDF; run commands from the release directory.

1 Part One: Backpropagation (35 points)

1.1 Introduction

Sparse MoE layers select a small number of feedforward experts for each token representation [1, 2]. Their components are familiar: fully connected layers, ReLU, Softmax, weighted sums, and residual connections. We first study a small, differentiable example. A local convolution supplies its context representation; Part Three replaces this encoder with a provided causal context module.

1.2 Network Details

For an input $\mathbf{s} \in \mathbb{R}^4$, a shared convolutional kernel $\mathbf{q} \in \mathbb{R}^3$ produces

$$a_j = \sum_{t=1}^{3} q_t s_{j+t-1}, \quad h_j = \mathrm{ReLU}(a_j), \qquad j = 1, 2. \tag{1}$$

There is no padding, the stride is one, and the operation is cross-correlation. This encoder gives $D = 2$; the downstream module is written for general D. Use $\mathrm{ReLU}'(0) = 0$. For E experts, hidden width H, and K prediction classes, define

$$\mathbf{v}_e = W_{1,e}\mathbf{h} + \mathbf{b}_{1,e}, \qquad \mathbf{u}_e = \mathrm{ReLU}(\mathbf{v}_e), \qquad \mathbf{f}_e = W_{2,e}\mathbf{u}_e + \mathbf{b}_{2,e}, \tag{2}$$

$$\mathbf{g} = \mathrm{Softmax}(G\mathbf{h}), \qquad \mathbf{r} = \mathbf{h} + \sum_{e=1}^{E} g_e \mathbf{f}_e, \qquad \widehat{\mathbf{y}} = \mathrm{Softmax}(C\mathbf{r} + \mathbf{b}_c). \tag{3}$$

Here $W_{1,e} \in \mathbb{R}^{H \times D}$, $W_{2,e} \in \mathbb{R}^{D \times H}$, $G \in \mathbb{R}^{E \times D}$, and $C \in \mathbb{R}^{K \times D}$. All biases have the corresponding output dimension; the router has no bias. For a mini-batch of m samples, use

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log \widehat{y}_k^{(i)}. \tag{4}$$

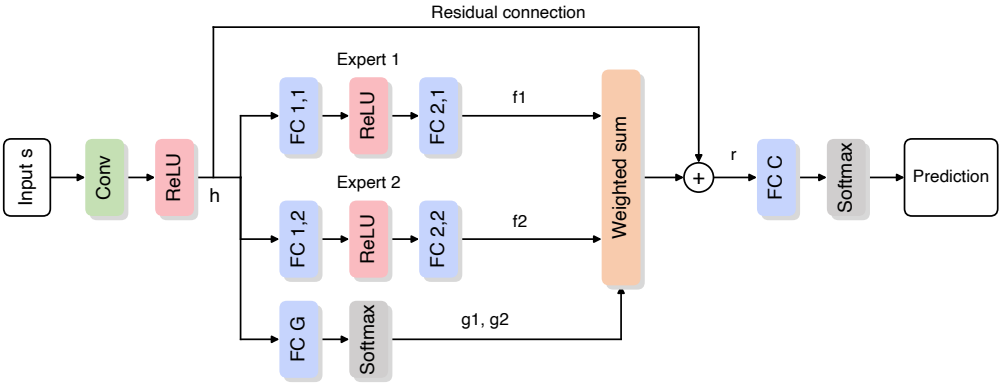


Figure 2: The handwritten example. Blue, red, gray, and orange boxes denote FC, ReLU, Softmax, and weighted summation; the circle denotes addition. Both experts are selected.

For numerical computations, use $m = 1$, $D = H = E = 2$, $K = 3$, and the following parameters. Unlisted expert biases are zero.

$$\mathbf{s} = (2, 1, -1, 3)^\top, \quad \mathbf{q} = (1, \tfrac{1}{2}, \tfrac{1}{2})^\top, \quad \mathbf{y} = (0, 1, 0)^\top,$$

$$W_{1,1} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad W_{1,2} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad \mathbf{b}_{1,1} = \begin{pmatrix} 0 \\ 1/2 \end{pmatrix}, \quad \mathbf{b}_{1,2} = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

$$W_{2,1} = \begin{pmatrix} 1/5 & 0 \\ 0 & 1/5 \end{pmatrix}, \quad W_{2,2} = \begin{pmatrix} 0 & 1/5 \\ 1/5 & 0 \end{pmatrix}, \quad \mathbf{b}_{2,1} = \begin{pmatrix} 0 \\ 1/2 \end{pmatrix}, \quad \mathbf{b}_{2,2} = \begin{pmatrix} 1/2 \\ 0 \end{pmatrix},$$

$$G = \begin{pmatrix} 1/4 & -1/4 \\ 0 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 1/5 & 0 \\ 0 & 1/5 \\ 1/5 & 1/5 \end{pmatrix}, \quad \mathbf{b}_c = (-0.49, -0.49, -0.98)^\top. \tag{5}$$

1.3 Task Description

Please show necessary derivations and variable dimensions. A **vectorial form** is highly encouraged; intermediate adjoints may be reused.

1. 📝 Compute $\mathbf{h}$, both expert outputs, $\mathbf{g}$, $\mathbf{r}$, the prediction, and the loss for Eq. (5). (**6 points**)
2. 📝 Derive the Softmax Jacobian and the gradient of cross-entropy with respect to prediction logits. Explain how mini-batch averaging enters the gradient. (**6 points**)
3. 📝 For general m, D, H, E, K, derive gradients of $W_{1,e}$, $\mathbf{b}_{1,e}$, $W_{2,e}$, $\mathbf{b}_{2,e}$, G, C, $\mathbf{b}_c$, and each input $\mathbf{h}$. Explicitly include the **expert, router, and residual paths**. Evaluate only $\partial\mathcal{L}/\partial G_{1,1}$ numerically. (**16 points**)
4. 📝 Backpropagate through the shared convolution. You may use the checked upstream gradient $\nabla_{\mathbf{h}}\mathcal{L} = (349, -173)^\top/2400$. Compute $\nabla_{\mathbf{q}}\mathcal{L}$, $\partial\mathcal{L}/\partial s_2$, and one kernel update with learning rate 0.1. Give mini-batch SGD pseudocode for all trainable parameters. (**7 points**)

2 Part Two: Implementing the Experts (20 points)

2.1 Introduction

Implement the module in Eqs. (2)–(4) using `numpy`. Automatic differentiation is **not allowed**. Complete `NumPy/model.py`; data, initialization, a central-difference checker, and the training loop are provided. Batched samples are stored in rows; parameter arrays follow the column-vector convention above.

2.2 Sparse Routing and Gradient Verification

For k selected experts, let $S(\mathbf{h})$ contain the indices of the k largest router logits and replace the gate in Eq. (3) by

$$g_e = \begin{cases} \dfrac{\exp((G\mathbf{h})_e)}{\sum_{j\in S(\mathbf{h})}\exp((G\mathbf{h})_j)}, & e \in S(\mathbf{h}), \\ 0, & e \notin S(\mathbf{h}). \end{cases} \tag{6}$$

The selected set is fixed locally. The checker supplies masks away from Top-k ties and ReLU kinks; derivatives of discrete selection are not required. It uses $\epsilon = 10^{-6}$ and reports the maximum normalized error

$$\max_j \frac{|a_j - n_j|}{\max(1, |a_j| + |n_j|)}, \qquad n_j = \frac{\mathcal{L}(\theta + \epsilon\mathbf{e}_j) - \mathcal{L}(\theta - \epsilon\mathbf{e}_j)}{2\epsilon}. \tag{7}$$

2.3 Task Description

1. Implement forward and backward propagation, including the prediction head and shared input gradient. Reproduce Part One and support the provided three-expert, Top-2 case. (**10 points**)
2. Run `python NumPy/run.py -check`. Report errors for every parameter family and the input; aim for errors below 10^{-5}. Diagnose the supplied faulty input-gradient variant and identify the missing computational path. (**6 points**)
3. Run `python NumPy/run.py -train` on the provided 128-example batch from the same corpus. Use 400 SGD updates with learning rate 0.2; plot loss and show three predictions. Context features are fixed. No hyperparameter search or generalization target is required. (**4 points**)

3 Part Three: A Small MoE Language Model (45 points)

3.1 Dataset

Predict the next byte in a fixed mixture of Tiny Shakespeare [4] and CPython source code [5]. The vocabulary consists of the 256 possible bytes. The package supplies tokenized data and fixed splits:

Shakespeare uses contiguous 80/10/10 percent partitions; code is split by file. Windows never cross file or split boundaries. Training samples the two domains equally. Domain labels are used only for evaluation, never as router inputs.

3.2 Network Details

The provided two-block causal model uses dimension $D = 64$, four attention heads, and context length $T = 64$. Its causal context module uses only the current and preceding bytes. Embeddings, normalization, attention, residual wrappers, optimization, and load balancing are provided; their internals are not assessed. Complete only the marked core methods in `MoE/model.py`; PyTorch automatic differentiation is allowed here.

Each expert maps $64 \to 128 \to 64$ with ReLU. A block uses $\mathbf{x} + \mathrm{MoE}(\mathrm{LN}(\mathbf{x}))$, with Eq. (6), $E = 16$, and $k = 2$. Part One omits the normalization wrapper for hand calculation. This is a teaching-scale version of token-level FFN routing, rather than a reproduction of a pretrained large model.

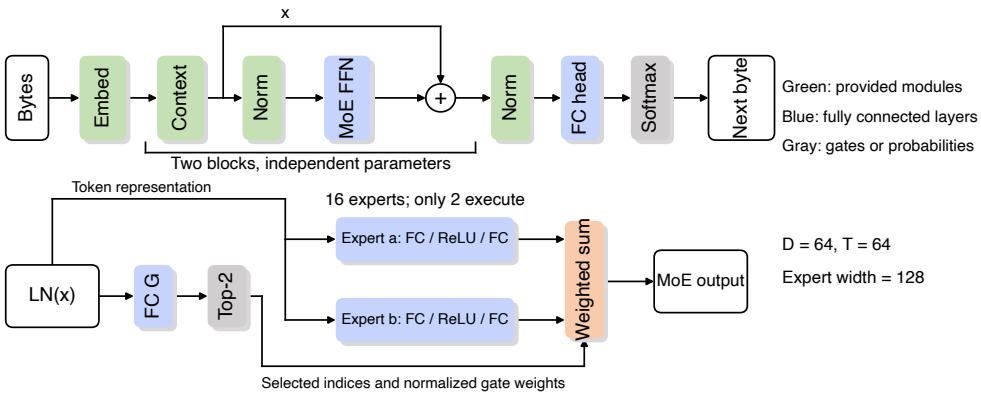


Figure 3: The training model. The two blocks share an architecture but have independent parameters. Only the selected token–expert pairs execute the expert MLPs.

Table 1: Three configurations; all other model components are identical. The parameter match is approximate (within 1%).

Configuration	Experts	Selected	FFN hidden width
Dense, computation matched	1	1	264
Sparse MoE	16	2	128 per expert
Dense, parameter matched	1	1	2056

3.3 Training and Evaluation

Use seed 416, 3000 updates, and batches of 32 sequences of length 64. Keep the provided optimizer, schedule, data order, and checkpoint rule: lowest validation cross-entropy. The auxiliary balancing term has coefficient 0.01 and requires no manual derivation. Evaluate held-out nonoverlapping full-length packs. Report each domain separately and use

$$\mathcal{L}_{\text{test}} = \tfrac{1}{2}(\mathcal{L}_{\text{text}} + \mathcal{L}_{\text{code}}), \qquad \text{PPL} = \exp(\mathcal{L}_{\text{test}}) \tag{8}$$

as the main metric. Byte-level perplexity is comparable only under this common tokenization and protocol. This comparison fixes updates and data order, not wall-clock time or convergence. Credit depends on correctness and analysis, not a MoE win or an absolute speed/PPL target; small numerical differences across devices are expected.

3.4 Task Description

1. Implement the expert forward pass, router, and selected-token dispatch. First run the provided CPU check, `python MoE/diagnostics.py -check`; then execute `python MoE/train.py -mode all`. Report training/validation curves and test metrics for all three configurations. Do not compute all experts and then mask their outputs. (**10 points**)
2. Compare quality, total parameters, matrix MACs per token, and full-batch forward latency ($B = 32, T = 64$) on the same device. Include attention, router, experts, and the head in MACs; include dispatch in timing. Run `python MoE/diagnostics.py`: paired routes are exchanged across sequences **at the same position** in the last layer, preserving expert–token counts. It checks prefix invariance and reports diagnostic CE over three fixed shuffle seeds. This batch-dependent perturbation is not standalone LM performance; do not infer a causal training benefit. Use one table and one figure to discuss capacity, computation, routing, and expert usage. Usage alone does not establish semantic specialization; lower MACs need not imply lower latency. (**20 points**)
3. 🤖 **Design your own small investigation.** Choose a testable question about sparse computation, routing, or model behavior. AI tools may assist with ideas, code, and figures. (**15 points**)

   Optional starting points (other questions are equally welcome): **(a)** compare inference $k = 1, 2, 4$ at one Top-2 checkpoint: quality versus MACs; **(b)** keep the last layer's selected experts fixed and compare learned versus equal gate weights $(1/2)$: is selecting experts enough?

   Define your question and a meaningful control; gather evidence from a few experiments or analyses anchored in the required results. Explain your interpretation and a limitation; improvements are not required. Limit the investigation to 600 English words or 1000 Chinese characters, and at most two figures or tables; provide runnable code and the AI Use Statement. Credit is based on the question (3), design (4), evidence (4), interpretation (3), and disclosure (1).

4 Submit Format

Submit `{StudentID}_{Name}_hw1.zip`, containing:

1. A PDF answering every numbered item, including handwritten photographs for Part One, gradient-check results, experimental comparisons, and the exploration.
2. Runnable core code, configurations, and compact result logs for Parts Two and Three. Include necessary references and assistance disclosures; full model checkpoints and the supplied dataset are not required.

AI Use Statement

Following ICLR, end your report with 3–5 sentences disclosing tools/models, affected items/tasks, and verification, or explicitly state no AI use. Confirm compliance with the rules above and responsibility for your submission. This statement is outside the 600-word/1000-character limit.

References

[1] W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *JMLR*, 23(120), 2022.

[2] A. Q. Jiang et al. Mixtral of Experts. *arXiv:2401.04088*, 2024.

[3] DeepSeek-AI et al. DeepSeek-V3 Technical Report. *arXiv:2412.19437*, 2025.

[4] A. Karpathy. Tiny Shakespeare corpus, `char-rnn`. `https://github.com/karpathy/char-rnn`.

[5] Python Software Foundation. CPython 3.10.16 source distribution. `https://www.python.org/downloads/release/python-31016/`.